Reproducible Data Analysis

Stat 184 : Hw 4.3

Mia Mason

2026-04-14

Table of contents

1 Busiest Airports Analysis **2**
1.1 Narrative 2
1.2 Table and Plot 2

2 Monte Carlo Numerical Integration **5**
2.1 Narrative 5
2.2 Small Multiple Visual 5

3 Planning and Prompting GenAI Tools **7**
3.1 Plan Informed Prompt and Response 7
3.2 Generic Prompt and Response 9
3.3 Comparing and Contrasting the Responses from the Planned and Generic Prompts 10

4 Reflection **11**

5 GenAI Usage Appendix **13**
5.1 ChatGPT Prompts and Responses 13
5.2 ASU Image Accessibility Generator Prompts and Responses 26

6 Code Appendix **31**

1 Busiest Airports Analysis

1.1 Narrative

Across both the table and the graph (Table 1 and Figure 1), the most noticeable story that can be learned is how air travel has recovered since the height of the COVID-19 pandemic that took place in 2020.

The only United States based airport in the data is the Hartsfield-Jackson Airport in Atlanta, Georgia (ATL). Looking at the graph and the table, it stands out from the rest of the airports because of how much passenger traffic increased from 2020 onward. ATL is known to be a hub for domestic flights within the United States, and it's quick recovery after COVID-19 can be seen through the numbers on the table and graph. This can be contrasted with other international airports like Frankfurt (FRA) and Dubai (DXB) which had a less impressive, but gradual, increase in traffic, most likely due to their main market being international travel, which had much stricter rules in place that only gradually lifted as the years went on.

East Asian countries like Korea and China, with respective airports in the data being Incheon (ICN), Beijing (PKX), and Shanghai (PVG), had even stricter rules regarding travel during the pandemic. Looking at the graph, it can be easy to see how tight those restrictions really were as they had delayed recovery in passenger traffic as compared to the other global airports covered in the table and graph.

Overall, the data in the table and graph shows that the recovery of air travel, domestic and international, eventually recovered. The rate in which the recovery happened did differ depending on regional policies.

1.2 Table and Plot

Table 1: Airport traffic across six airports, 2020-2024

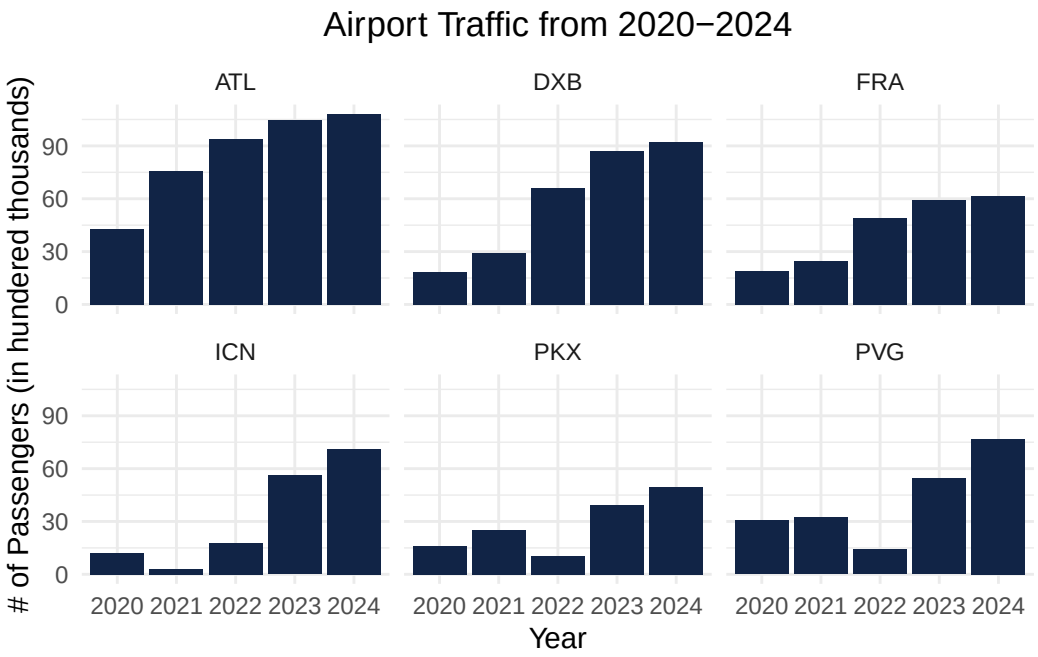
Year	Atlanta	Frankfurt	Beijing	Incheon	Shanghai	Dubai
2020	42,918,685	18,768,601	16,091,449	12,094,851	30,476,531	18,229,461
2021	75,704,760	24,814,921	25,051,012	3,198,909	32,206,814	29,110,609
2022	93,699,630	48,923,474	10,277,623	17,869,759	14,178,385	66,069,981
2023	104,653,451	59,359,539	39,410,776	56,131,064	54,496,397	86,994,365
2024	108,067,766	61,564,957	49,441,029	71,156,947	76,787,039	92,300,000

Alt-text: Table of statistics for Atlanta, Frankfurt, Beijing, Incheon, Shanghai, and Dubai airports from 2020 to 2024.

Long Description: The image is a table displaying yearly data from 2020 to 2024, showing figures for six airports: Atlanta, Frankfurt, Beijing, Incheon, Shanghai, and Dubai. Each year

is followed by corresponding numbers for each city, representing a dataset that is related to the number of passengers at those airports. Rows represent years, while columns contain the location in which an airport is located followed by numeric values increasing over time. The table is organized with headers for each column.

Figure 1: Plot visualization for airport traffic table



Alt-text: Bar graphs of airport traffic from 2020 to 2024 for ATL, DXB, FRA, ICN, PKX, and PVG.

Long Description: The image is a collection of bar graphs showing airport traffic from 2020 to 2024 for six different airports: ATL, DXB, FRA, ICN, PKX, and PVG. Each graph displays the number of passengers in hundreds of thousands on the y-axis and the year from 2020 to 20204 on the x-axis. The bars are dark blue against a light background with grid lines for reference. ATL shows a steady increase, peaking around 2023 and 2024. DXB's traffic rises sharply, leveling off in 2023 and 2024. FREA sees gradual growth, while ICN has a significant surge in 2023 and 2024. PKX and PVG display moderate increase, with notable growth toward 2024.

2 Monte Carlo Numerical Integration

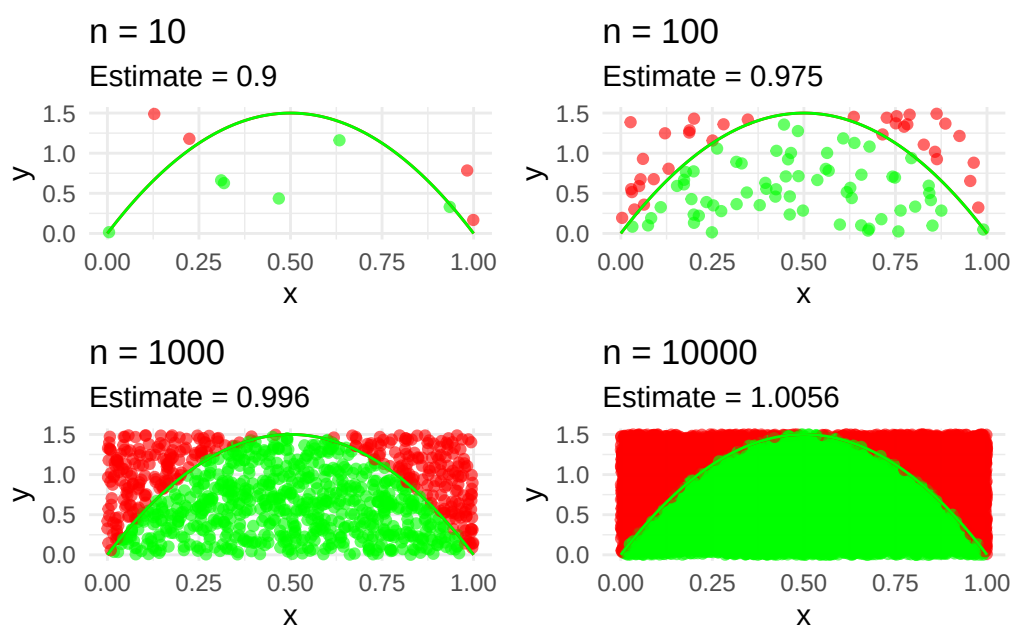
2.1 Narrative

The small multiple in Figure 2 shows how the Monte Carlo integration stabilizes as sample size increases. The smallest sample size, n = 10, has a highly variable estimate because the random points don't accurately represent the curve. By the largest sample size, n = 10,000, the plot is very densely populated but the points uniformly fill the rectangle and make the difference between the points that lay above and on/below the curve highly visible. The green region, being the points on and below the curve, approximate the true shape of the function better, revealing how the randomness once seen in the n = 10 sample size, averages out once the sample size becomes larger.

Looking at the largest sample size, n = 10,000, it would be a best guess to say that the exact value of the integral is 1. This is supported by simply looking at all the visuals together. When looking at each estimation generated by the sample size, it is apparent that as the sample size increases, the estimated value begins to approach a value of 1.

2.2 Small Multiple Visual

Figure 2: Small multiple visual for Monte Carlo Numerical Integration



Alt-text: Four scatter plots showing parabolic data distribution with varying sample sizes, n = 10, 100, 1000, 10000.

Long Description: The image consists of four scatter plots arranged in a 2x2 grid, illustrating the relationship between x and y coordinates with different sample sizes. Each plot features a green parabolic curve alongside red and green data points. The top-left plot (n=10) has sparse green points near the curve. The top-right plot (n=100) shows more densely distributed points, with some red points surrounding the green curve. In the bottom-left plot (n=1000), data points are densely packed with a clear distinction between red and green regions. The bottom-right plot (n=10000) is densely filled with red and green points, illustrating a high level of data concentration. All plots share labeled axes with "x" and "y" indicators and grid lines.

3 Planning and Prompting GenAI Tools

3.1 Plan Informed Prompt and Response

Plan and goal for tidying and wrangling calcium data

Goals:

- wrangle the calcium.csv table to explore the difference in ulnar calcium measurements across time and treatment groups

Plan:

1. load tidyverse package into rstudio

2. rename columns of the table to the initial, year 1, year 2, and year 3 of each treatment group.

2.1 The first four columns should be labled initialA, year1A, year2A, year3A to indicate the first treatment group A which is the control group that did not recieve any treatment and is made up of 15 women

2.2 The second four columns should be labled initialB, year1B, year2B, to indicate the second treatment group B which is the group that recieved treatment and is made up of 16 women

3. use pivot_longer() to make only 4 columns for initial, year1, year2, and year3, but adds an extra column to indicate which treatment group the person is a part of. There should be a total of 5 columns, treatment_group, initial, year1, year2, and year3.

4. Filter out any row that has N/A, eg. the extra row in treatment group A that has N/A's because there is one less participant in group A than group B

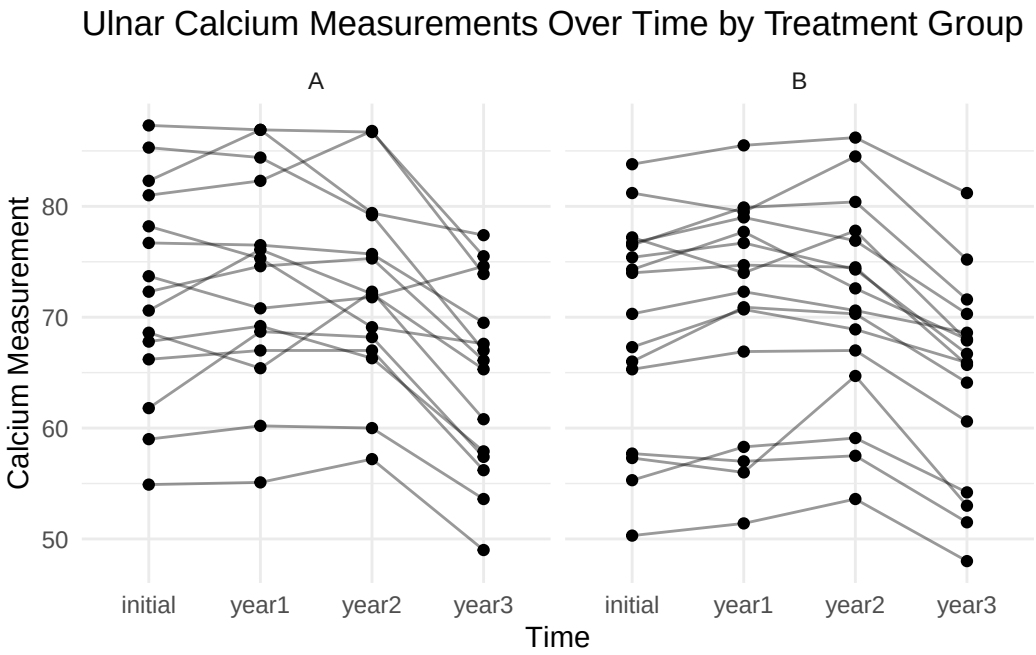
5. Use ggplot2 to create a graph/plot that will help explore the difference in ulnar calcium measurements across time and the two treatment groups.

5.1 Facet the graph/plot by the treatment groups A and B

5.2 Make the x-axis the initial reading, and year 1, year 2, year 3

5.3 Make the y-axis the calcium measurements

Figure 3: Plot generated by ChatGPT using a plan prompt



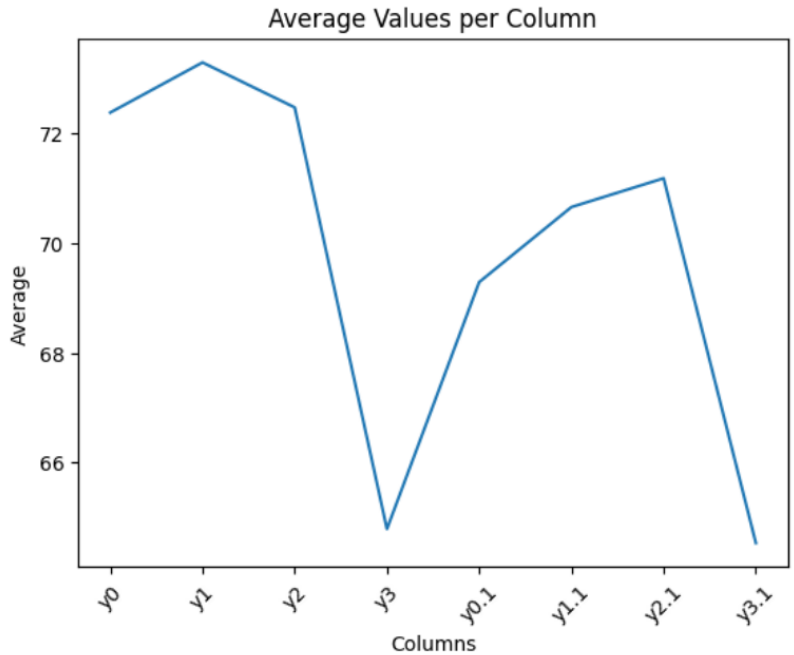
Alt-text: Line graph of ulnar calcium measurements over time by treatment group A and B.

Long Description: The image is a line graph displaying "Ulnar Calcium Measurements Over Time by Treatment Group." It consists of two sections labeled A and B, represenging different treatment groups. Each section contains multiple lines connectin data points across four time intervals marked as initial, year 1, year 2, and year 3. The y-axis represents "Calcium Measurement" with values ranging fro 50 to 90. Group A shows more variability with lines intersecting and diverging, while group B lines appear more parallel, indicating consistent measurements over time. Both sections illustrate individual trends with black dots making measurement points connected by gray lines.

3.2 Generic Prompt and Response

Prompt: Create a data visualization for these data. Calcium.csv

Figure 4: Plot generated by ChatGPT using a generic prompt



Alt-text: Line graph showing average values with fluctuations across columns.

Long Description: The image shows a lone graph titled "Average Values per Column." It depicts a blue line chart illustrating average values across various columns labeled on the x-axis. The graph features nine data points, with the line demonstrating fluctuations in value. The line starts at 72, peaks, sharply drops to a low point, rises again, and then slightly decreases. The y-axis is labeled "Average," with values ranging from 66 to 72 in increments of two. The x-axis is labeled "Columns," with labels from "y0" to "y3.1."

ChatGPT Response: I created a visualization showing the average value of each column in your dataset. What this shows: • Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). • You can quickly compare which variables tend to have higher or lower values overall. • For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create:

- Boxplots (to show spread and outliers)
- Time series plots (if rows represent time/order)

- Correlation heatmaps
- Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me

Prompt: What is your version information?

ChatGPT Response: You're chatting with ChatGPT based on the GPT-5.3 architecture.

Key details:

- Model: GPT-5.3
- Knowledge cutoff: August 2025
- Current date (system): March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know

(This chat took place on Tuesday, March 31st, 2026.)

3.3 Comparing and Contrasting the Responses from the Planned and Generic Prompts

The differences between both the conversations had with the AI tools and the results they produced are very apparent. Both the quality of the prompts used and the produced graphs vary, with the higher quality plan informed conversation yielding a graph that contained more detail and told more of the story presented by the data. The generic prompt did not include any context about the data or how the data should be organized into the graph, leaving the AI tool with more space to veer off track and include, or exclude, important details that would have aided in the understanding of the data.

Looking at the visualizations only, there is a huge loss in data as seen in the generic prompt graph. The AI tool decided that taking the average of all 31 cases, but still keeping duplicates of the initial, year 1, year 2, and year 3 time points was the most effective way at presenting the data. This makes sense, since the AI was not given any context about the data, all it had to go off of was the column names and values under them.

Overall, this section does a great job at showing the precautions people should take when they decide to utilize the help of an AI tool. There's no way for a computer to know everything that you want it to do if you don't explicitly tell it, in great detail, what it is that it should do. No matter how advanced these tools become in the future, it will never be completely human and so it's the humans job to double, triple, or even quadruple check, that what it is generating is accurate and correct.

4 Reflection

My time in this class over the past semester has taught me many things. Not only have I learned the basics of the R language, but how it is used in data analysis. This class even helped me build skills like critical thinking and problem solving, which honestly was something that I was struggling with when coming into the semester. As expected when beginning the semester, I was unfamiliar with R and other concepts covered in the class. By the end, however, I found myself thinking more with that data analyst mindset and in more detail than I did before.

One aspect of the class that I found especially helpful was the idea of always going through examples or answering questions with a small group before coming together as a class. Working through problems with others forced me to participate, which is something I do not usually do, and also actively using the information I learned rather than just memorizing and following along. It allowed me to consider different approaches when hearing other peoples answers, and also figure out where my own thought process might've gone wrong. This alone really strengthened my critical thinking skills. During the visualizations unit where we were learning to read graphs and plots using the EPT's, I had a hard time figuring out how to identify the mapping. Hearing my table mates explain how they found the mapping of a specific graph eventually helped me learn how to identify it myself.

Throughout the semester, I also developed technical skills like PCIP, data wrangling, how to use packages and creating visualizations. When working on the data wrangling for the busiest airport portion of this document I realized how different my plan was as compared to the plans I made in the beginning of the class. Since I didn't know much about using packages and their specific functions back then, I didn't know how to explain what I wanted to be done in my plan. Looking at my busiest airport data wrangling plan now, I see how I'm able to include certain functions I want to use to get the data looking how I want it to so I can create the graph from it. I do think I still need to work on the whole PCIP process, since it's still unfamiliar to me that I actually have to write out my goals and steps, rather than just keeping it as a mental note in my head just for me to forget it later. The main thing I learned in this class that I will always be grateful for is the use of packages. In all my other coding language classes, we were only taught how code using the base language. Libraries like matlab and Pandas for python were only briefly mentioned, and when we were required to use them, we weren't even taught about any of the features they included. That was unlike what I learned in this class, where we spent entire units learning how to code using the Tidyverse and other packages because they are essential to data analysis.

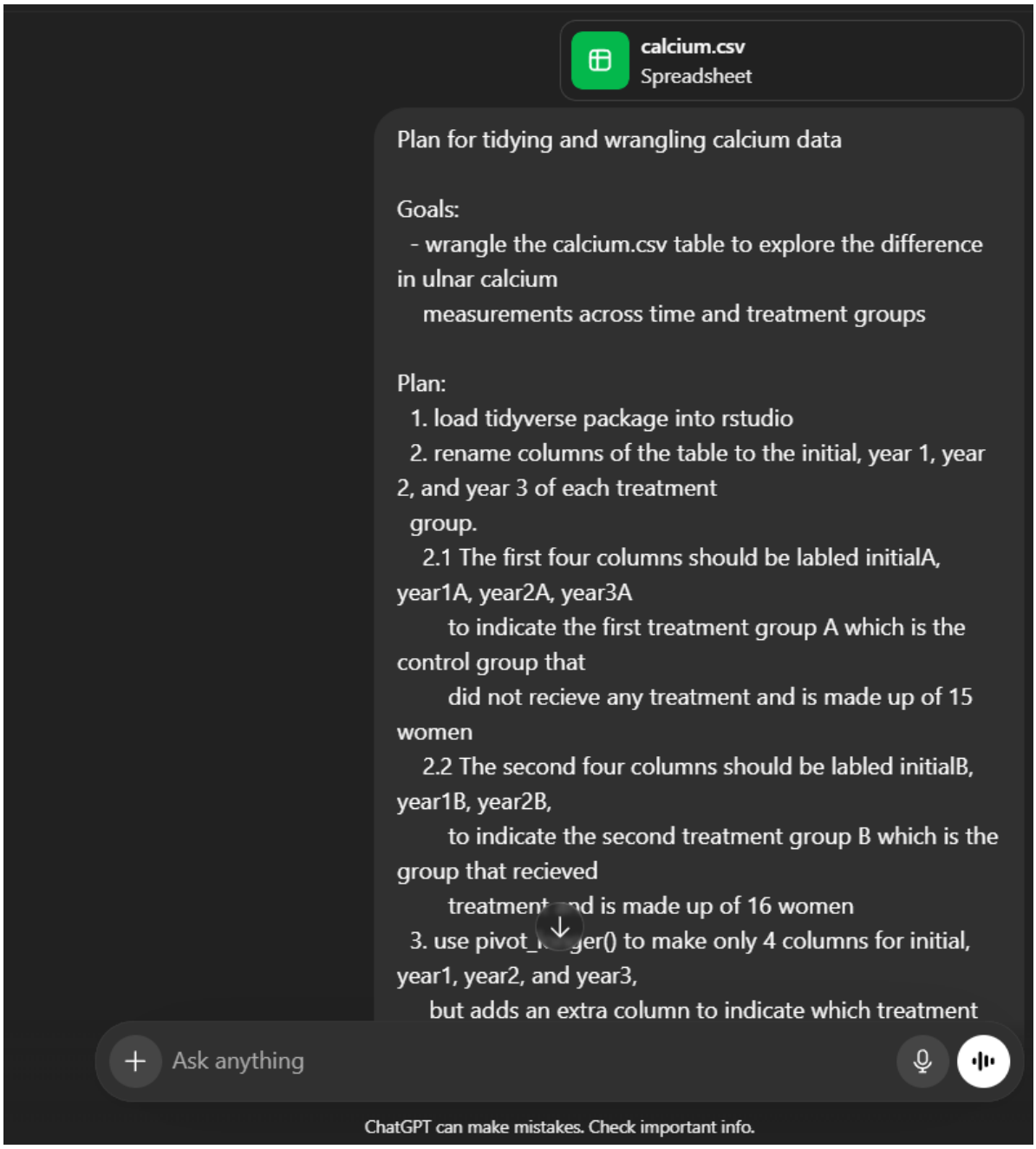
Aside from all these great things I learned from the class, I also recognized the areas in which I need to improve. The homework assignments were often very long and challenging, and there were times when I, very regretfully, did not complete them fully or submit them on time. While this was difficult, it also made me realize that I still need to work on my time management, as it is something that I am constantly struggling with. Another thing I realized I need to work on is persistence when facing problems that I run into. This is a huge area of concern for

me, as I tend to give up the moment something doesn't go right. However, I plan to take the experience from this class and use it as a way to grow moving forward.

As the semester reaches the end, I can see, and I'm proud of, the progress I have made in this class through my technical skills and learning approach. I plan to use all this new found knowledge in my future endeavors, like finally starting a personal project that I can use in resumes and internship/job application processes. I really enjoyed what I learned in this class, and I'm excited to learn even more.

5 GenAI Usage Appendix

5.1 ChatGPT Prompts and Responses



but adds an extra column to indicate which treatment group the person is a
part of. There should be a total of 5 columns, treatment_group, initial, year1,
year2, and year3.
4. Filter out any row that has N/A, eg. the extra row in treatment group A that
has N/A's because there is one less participant in group A than group B
5. Use ggplot2 to create a graph/plot that will help explore the difference in
ulnar calcium measurements across time and the two treatment groups.
5.1 Facet the graph/plot by the treatment groups A and B
5.2 Make the x-axis the initial reading, and year 1, year 2, year 3
5.3 Make the y-axis the calcium measurements

Your plan is already quite solid—you're thinking in the right "tidy data" direction. I'll tighten it up a bit, fix a couple inconsistencies, and give you a clean, **actionable wrangling plan + R code** that matches your stated goals.

✅ Refined Plan for Tidying / Wrangling Calcium Data

🎯 Goal

🎯 Goal

Reshape `calcium.csv` to analyze how **ulnar calcium measurements change over time** across **treatment groups A (control) and B (treatment)**.

🔧 Step-by-Step Plan

1. Load packages

- Use the **tidyverse** for data wrangling and visualization.

---

2. Rename columns clearly

- First 4 columns → Group A
  `initialA, year1A, year2A, year3A`
- Next 4 columns → Group B
  `initialB, year1B, year2B, year3B`
  ⚠️ (you were missing `year3B` in your plan—include it)

---

3. Reshape the data (pivot longer)

Instead of keeping wide format, convert to **long format** with:

- `treatment_group` (A or B)
- `time` (initial, year1, year2, year3)
- `calcium`

👉 This is better than your current idea (5 columns), because:

- It's **tidy format**
- Works naturally with **ggplot**
- Easier to compare trends

---

4. Remove missing values

- Drop rows with `NA` values (e.g., extra participant mismatch)

---

5. Create visualization

- X-axis → time (initial → year3)
- Y-axis → calcium
- Color → treatment group
- Facet → treatment group (optional but good for comparison)

💻 Clean R Code (Polished)

```R
library(tidyverse)

# 1. Load data
calcium <- read_csv("calcium.csv")

# 2. Rename columns
```

R

```r
# 2. Rename columns
colnames(calcium) <- c(
  "initialA", "year1A", "year2A", "year3A",
  "initialB", "year1B", "year2B", "year3B"
)

# 3. Pivot longer into tidy format
calcium_long <- calcium %>%
  pivot_longer(
    cols = everything(),
    names_to = c("time", "treatment_group"),
    names_pattern = "(initial|year1|year2|year3)(A|B)",
    values_to = "calcium"
  )

# 4. Remove missing values
calcium_long <- calcium_long %>%
  drop_na(calcium)

# 5. Convert time to ordered factor
calcium_long <- calcium_long %>%
  mutate(
    time = factor(time, levels = c("initial", "year1", "year2", "year3")),
    treatment_group = factor(treatment_group)
  )

# 6. Plot
ggplot(calcium_long, aes(x = time, y = calcium, group = interaction(row_number()
  geom_line(alpha = 0.4) +
  geom_point() +
  facet_wrap(~ treatment_group) +
  labs(
```

```r
labs(
  title = "Ulnar Calcium Measurements Over Time by Treatment Group",
  x = "Time",
  y = "Calcium Measurement"
) +
theme_minimal()
```

🔍 Key Improvements Over Your Original Plan

- ✅ Uses **true tidy format** (long instead of semi-wide)
- ✅ Fixes missing `year3B`
- ✅ Makes plotting much easier and more flexible
- ✅ Keeps treatment comparison clean and scalable

💡 Optional Enhancements

If you want deeper analysis:

- Add **mean trend lines:**

```r
stat_summary(aes(group = treatmen oup), fun = mean, geom = "line", size =
```

- Compare distributions:

- Compare distributions:

```r
geom_boxplot(aes(group = interaction(time, treatment_group)))
```

If you want, I can also help you:

- Interpret the plot (what trends mean scientifically)
- Add statistical tests (e.g., repeated measures / mixed models)
- Clean this into a submission-ready report format

There is an error in generating the plot:
Error in `geom_line()`:
! Problem while computing aesthetics.
ℹ Error occurred in the 1st layer.
Caused by error in `n()`:
! Must only be used inside data-masking verbs like `mutate()`, `filter()`, and `group_by()`.

Good catch—that error is a classic **ggplot + grouping issue**.

🔍 What went wrong

The problem is this part of your code:

```r
group = interaction(row_number(), treatment_group)
```

- `row_number()` internally relies on `n()`
- `n()` **only works inside dplyr verbs** like `mutate()` or `filter()`
- But `ggplot()` is **not** a data-masking context → so it throws that error

---

✅ Fix (Simple & Clean)

You need a **subject ID** to correctly draw lines across time for each person.

✔ Add an ID column *before* pivoting

```r
calcium <- calcium %>%
  mutate(id = row_number())
```

✔ Then update your pipeline

```r
calcium_long <- calcium %>%
```

R

```r
calcium_long <- calcium %>%
  pivot_longer(
    cols = -id,
    names_to = c("time", "treatment_group"),
    names_pattern = "(initial|year1|year2|year3)(A|B)",
    values_to = "calcium"
  ) %>%
  drop_na(calcium) %>%
  mutate(
    time = factor(time, levels = c("initial", "year1", "year2", "year3")),
    treatment_group = factor(treatment_group)
  )
```

Fix the plot

Now group by **id + treatment group**:

R

```r
ggplot(calcium_long, aes(x = time, y = calcium, group = interaction(id, treatmen
  geom_line(alpha = 0.4) +
  geom_point() +
  facet_wrap(~ treatment_group) +
  labs(
    title = "Ulnar Calcium Measurements Over Time by Treatment Group",
    x = "Time",
    y = "Calcium Measurement"
  ) +
  theme_minimal()
```

💡 Why this works

- Each participant gets a unique `id`
- `interaction(id, treatment_group)` ensures:
  - Lines connect **the same person over time**
  - No accidental mixing between subjects

🚀 Even Better (Optional Insight)

If you don't actually care about individual trajectories and just want trends, you can simplify:

```R
ggplot(calcium_long, aes(x = time, y = calcium, color = treatment_group)) +
  stat_summary(fun = mean, geom = "line", size = 1.2) +
  stat_summary(fun = mean, geom = "point") +
  labs(
    title = "Average Ulnar Calcium Over Time by Treatment Group",
    x = "Time",
    y = "Calcium Measurement"
  ) +
  theme_minimal()
```

If you want, I can ↪ help you interpret what the plot suggests about the treatment effect —that's usually the next step after getting this working

—that's usually the next step after getting this working.

Please print the entire fixed code

Here is the **complete, clean, fixed R script** from start to finish:

R

```r
# Load tidyverse
library(tidyverse)

# 1. Load data
calcium <- read_csv("calcium.csv")

# 2. Rename columns
colnames(calcium) <- c(
  "initialA", "year1A", "year2A", "year3A",
  "initialB", "year1B", "year2B", "year3B"
)

# 3. Add participant ID
calcium <- calcium %>%
  mutate(id = row_number())

# 4. Pivot to long format
calcium_long <- calcium %>%
  pivot_longer(
    cols = -id,
```

R

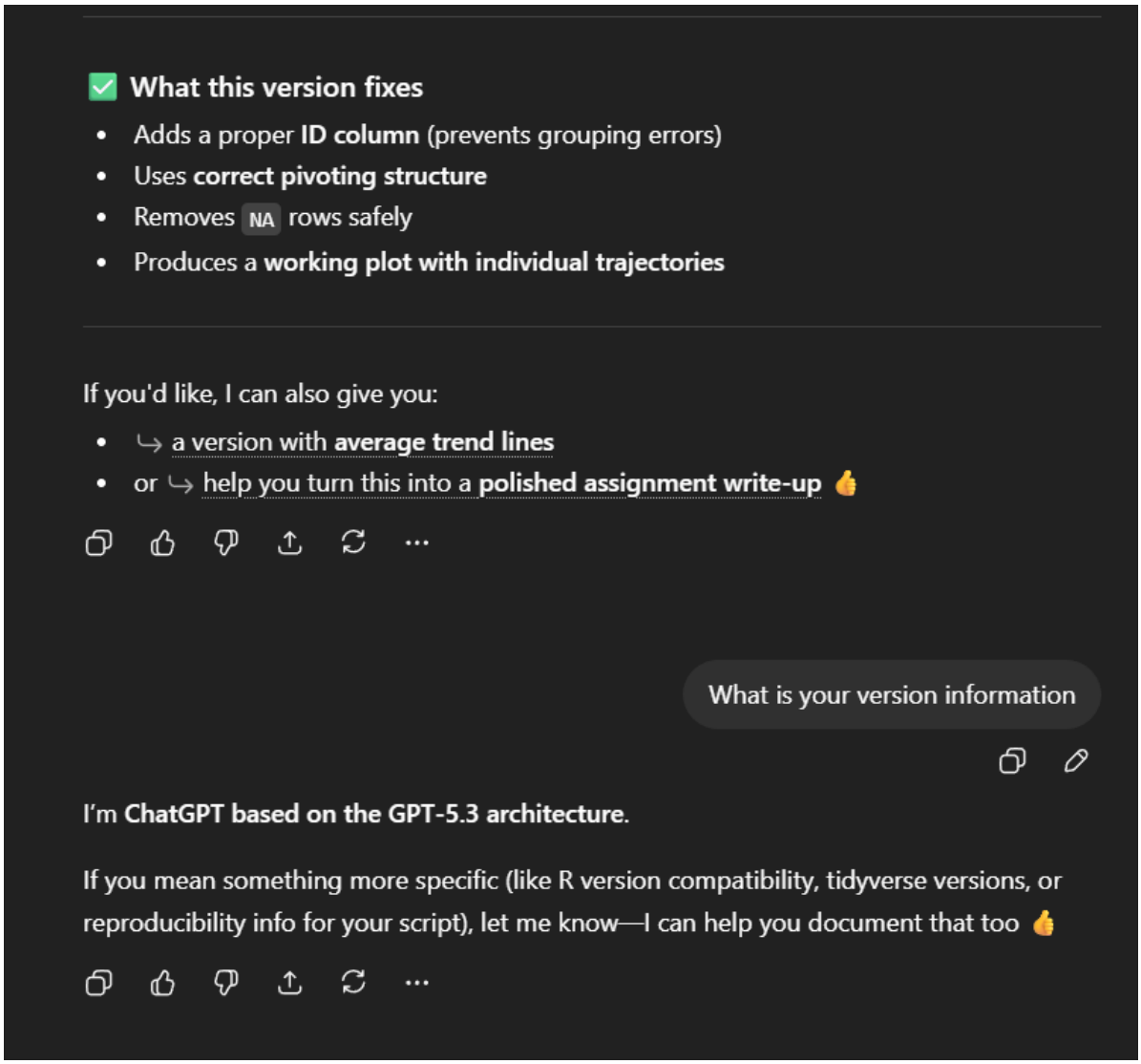
```r
    cols = -id,
    names_to = c("time", "treatment_group"),
    names_pattern = "(initial|year1|year2|year3)(A|B)",
    values_to = "calcium"
  ) %>%
  drop_na(calcium)

# 5. Clean and format variables
calcium_long <- calcium_long %>%
  mutate(
    time = factor(time, levels = c("initial", "year1", "year2", "year3")),
    treatment_group = factor(treatment_group)
  )

# 6. Plot individual trajectories
ggplot(calcium_long, aes(x = time, y = calcium, group = interaction(id, treatmen
  geom_line(alpha = 0.4) +
  geom_point() +
  facet_wrap(~ treatment_group) +
  labs(
    title = "Ulnar Calcium Measurements Over Time by Treatment Group",
    x = "Time",
    y = "Calcium Measurement"
  ) +
  theme_minimal()
```

✅ **What this version fixes**

- Adds a proper **ID column** (prevents grouping errors)



✅ **What this version fixes**

- Adds a proper **ID column** (prevents grouping errors)
- Uses **correct pivoting structure**
- Removes `NA` rows safely
- Produces a **working plot with individual trajectories**

If you'd like, I can also give you:

- ↳ a version with **average trend lines**
- or ↳ help you turn this into a **polished assignment write-up** 👍

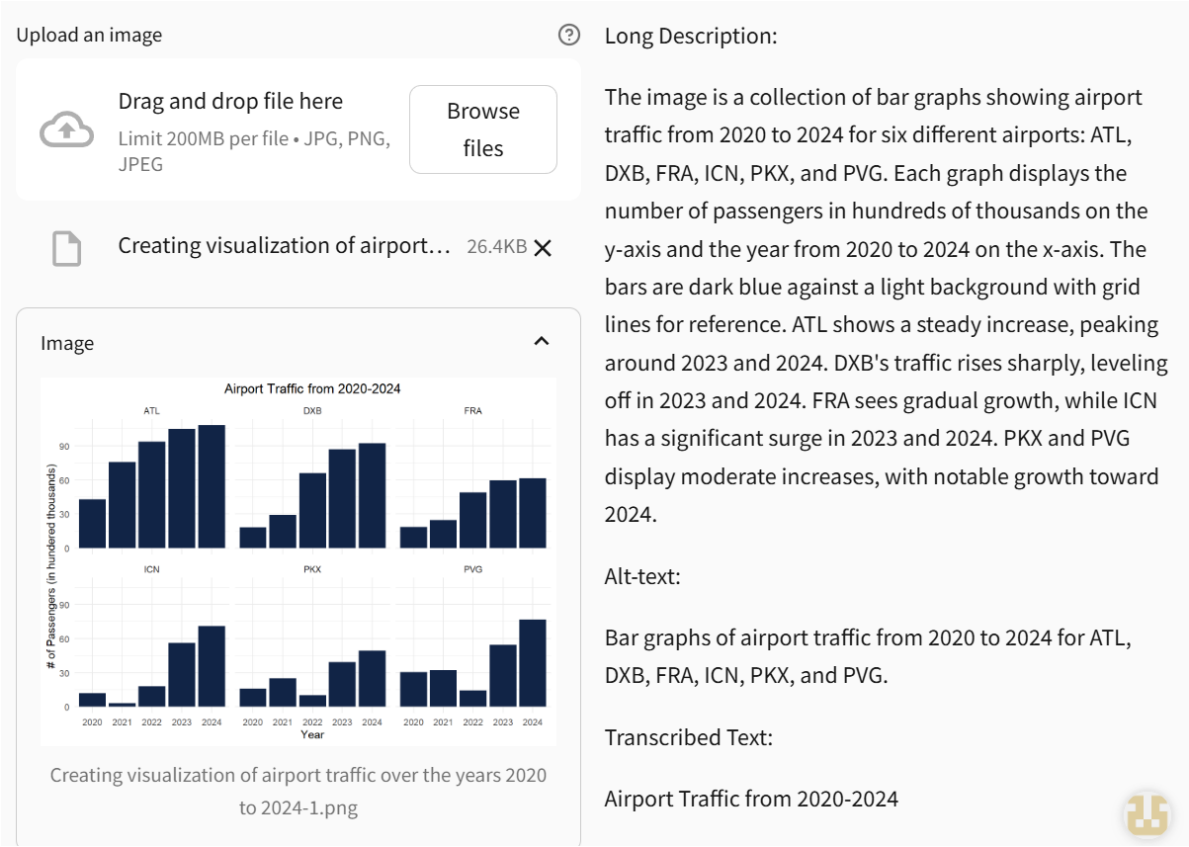
What is your version information

I'm **ChatGPT based on the GPT-5.3 architecture**.

If you mean something more specific (like R version compatibility, tidyverse versions, or reproducibility info for your script), let me know—I can help you document that too 👍

(This chat took place on April 14th, 2026)

5.2 ASU Image Accessibility Generator Prompts and Responses



Upload an image

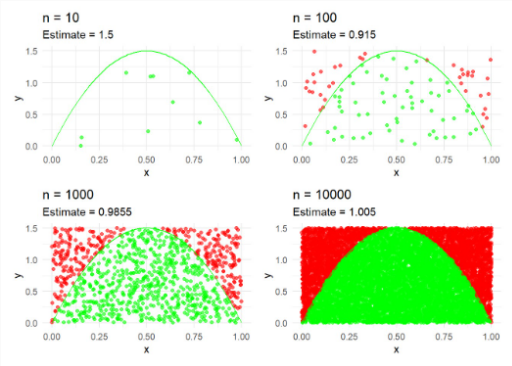
Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

Browse files

unnamed-chunk-1-1.png 171.1KB

Image



unnamed-chunk-1-1.png

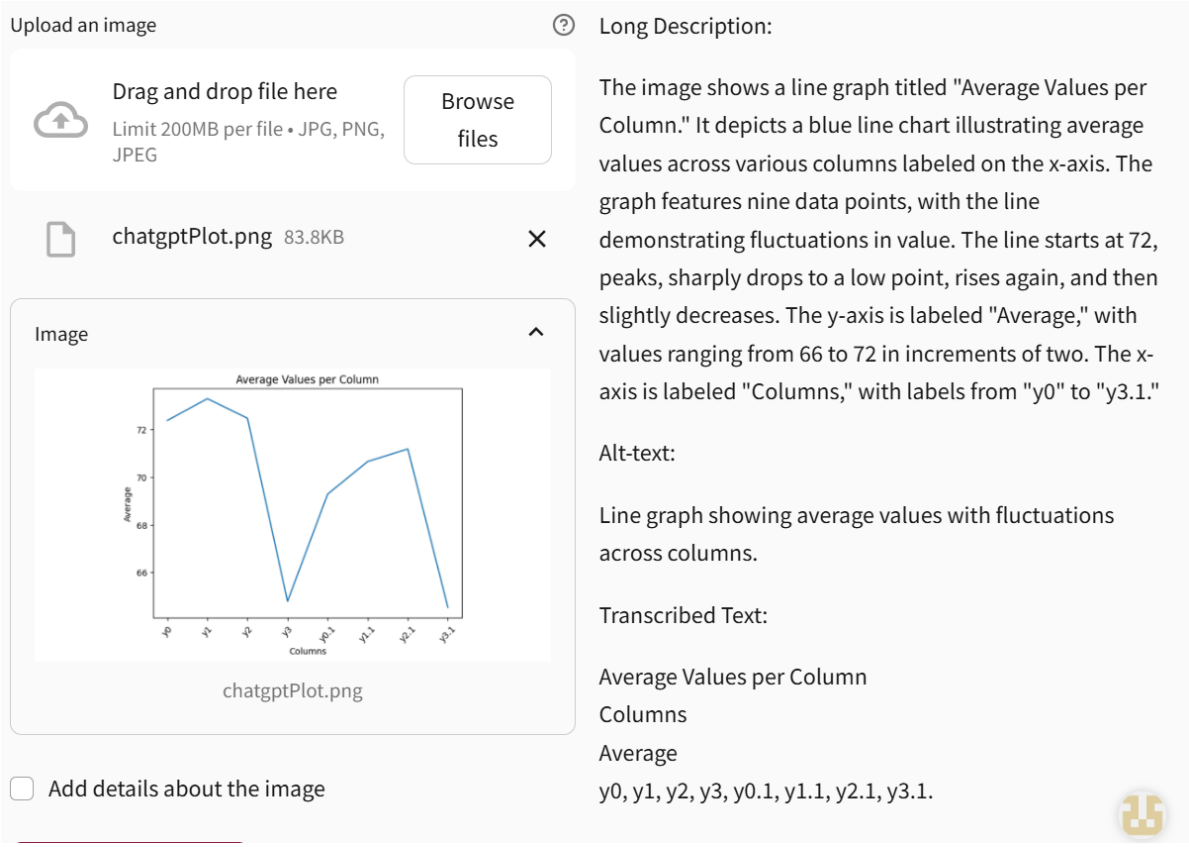
Long Description:

The image consists of four scatter plots arranged in a 2x2 grid, illustrating the relationship between x and y coordinates with different sample sizes. Each plot features a green parabolic curve alongside red and green data points. The top-left plot (n = 10) has sparse green points near the curve. The top-right plot (n = 100) shows more densely distributed points, with some red points surrounding the green curve. In the bottom-left plot (n = 1000), data points are densely packed with a clear distinction between red and green regions. The bottom-right plot (n = 10000) is densely filled with red and green points, illustrating a high level of data concentration. All plots share labeled axes with "x" and "y" indicators and grid lines.

Alt-text:

Four scatter plots showing parabolic data distribution with varying sample sizes, n = 10, 100, 1000, and 10000.

Transcribed Text:


Upload an image
Drag and drop file here
Limit 200MB per file • JPG, PNG, JPEG
Browse files
chatgptPlot.png 83.8KB
Image
Average Values per Column
Average
72
70
68
66
y0
y1
y2
y3
y0.1
y1.1
y2.1
y3.1
Columns
chatgptPlot.png
Add details about the image
Long Description:
The image shows a line graph titled "Average Values per Column." It depicts a blue line chart illustrating average values across various columns labeled on the x-axis. The graph features nine data points, with the line demonstrating fluctuations in value. The line starts at 72, peaks, sharply drops to a low point, rises again, and then slightly decreases. The y-axis is labeled "Average," with values ranging from 66 to 72 in increments of two. The x-axis is labeled "Columns," with labels from "y0" to "y3.1."
Alt-text:
Line graph showing average values with fluctuations across columns.
Transcribed Text:
Average Values per Column
Columns
Average
y0, y1, y2, y3, y0.1, y1.1, y2.1, y3.1.

Upload an image

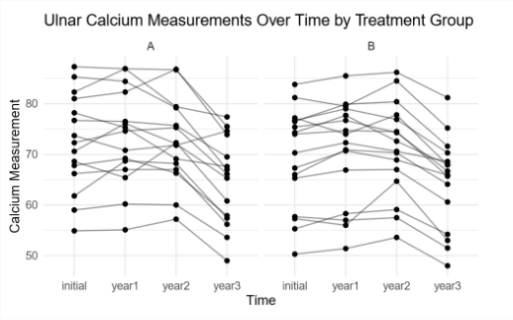
Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

Browse files

Screenshot 2026-04-14 192357… 102.1KB

Image



Screenshot 2026-04-14 192357.png

Add details about the image

Long Description:

The image is a line graph displaying "Ulnar Calcium Measurements Over Time by Treatment Group." It consists of two sections labeled A and B, representing different treatment groups. Each section contains multiple lines connecting data points across four time intervals marked as initial, year 1, year 2, and year 3. The y-axis represents "Calcium Measurement" with values ranging from 50 to 90. Group A shows more variability with lines intersecting and diverging, while Group B lines appear more parallel, indicating consistent measurements over time. Both sections illustrate individual trends with black dots marking measurement points connected by gray lines.

Alt-text:

Line graph of ulnar calcium measurements over time by treatment group A and B.

Transcribed Text:

Ulnar Calcium Measurements Over Time by Treatment

Upload an image

Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

Browse files

Screenshot 2026-04-14 192841…. 31.3KB

Image

Year	Atlanta	Frankfurt	Beijing	Incheon	Shanghai	Dubai
2020	42,918,685	18,768,601	16,091,449	12,094,851	30,476,531	18,229,461
2021	75,704,760	24,814,921	25,051,012	3,198,909	32,206,814	29,110,609
2022	93,699,630	48,923,474	10,277,623	17,869,759	14,178,385	66,069,981
2023	104,653,451	59,359,539	39,410,776	56,131,064	54,496,397	86,994,365
2024	108,067,766	61,564,957	49,441,029	71,156,947	76,787,039	92,300,000

Screenshot 2026-04-14 192841.png

Add details about the image

Long Description:

The image is a table displaying yearly data from 2020 to 2024, showing figures for six cities: Atlanta, Frankfurt, Beijing, Incheon, Shanghai, and Dubai. Each year is followed by corresponding numbers for each city, representing a dataset that appears related to city statistics such as population counts or another measurement. Rows represent years, while columns contain city names followed by numeric values creasing over time. The table is organized with headers for each column.

Alt-text:

Table of statistics for Atlanta, Frankfurt, Beijing, Incheon, Shanghai, and Dubai from 2020 to 2024.

Transcribed Text:

(These images descriptions were generated on April 14th, 2026 using the ASU Image Accessibility Generator)

6 Code Appendix

```
library(tidyverse)
library(rvest)
library(kableExtra)
library(knitr)

# ATL raw and extracted traffic data
ATLRawList <- read_html(
  x = "https://en.wikipedia.org/wiki/Hartsfield%E2%80%93Jackson_Atlanta_International_Airport
) |>
  html_elements(css = "table") |>
  html_table()

ATLTraffic <- bind_cols(ATLRawList[[11]])

# FRA raw and extracted traffic data
FRARawList <- read_html(
  x = "https://en.wikipedia.org/wiki/Frankfurt_Airport#Annual_traffic"
) |>
  html_elements(css = "table") |>
  html_table()

FRATraffic <- bind_cols(FRARawList[[14]])

# PKX raw and extracted traffic data
PKXRawList <- read_html(
  x = "https://en.wikipedia.org/wiki/Beijing_Daxing_International_Airport#Statistics"
) |>
  html_elements(css = "table") |>
  html_table()

PKXTraffic <- bind_cols(PKXRawList[[9]])

# ICN raw and extracted traffic data
ICNRawList <- read_html(
  x = "https://en.wikipedia.org/wiki/Incheon_International_Airport#Traffic_and_statistics"
) |>
  html_elements(css = "table") |>
  html_table()
```

```r
ICNTraffic <- bind_cols(ICNRawList[[10]])

# PVG raw and extracted traffic data
PVGRawList <- read_html(
  x = "https://en.wikipedia.org/wiki/Shanghai_Pudong_International_Airport#Statistics"
) |>
  html_elements(css = "table") |>
  html_table()

PVGTraffic <- bind_cols(PVGRawList[[7]])

# DXB raw and extracted traffic data
DXBRawList <- read_html(
  x = "https://en.wikipedia.org/wiki/Dubai_International_Airport#Statistics"
) |>
  html_elements(css = "table") |>
  html_table()

DXBTraffic <- bind_cols(DXBRawList[[7]])

# ATL Traffic wrangling
ATLTraffic <- ATLTraffic |>
  rename(
    year = ...1,
    passengers = Passengers
  ) |>
  select(
    year,
    passengers
  ) |>
  slice_head(n=26) |>
  mutate(
    passengers = readr::parse_number(passengers, na = c(",", "NA"))
  )

# FRA Traffic wrangling
FRATraffic <- FRATraffic |>
  rename(
    year = Year,
    passengers = Passengers
  ) |>
  select(
```

```r
    year,
    passengers
  ) |>
  mutate(
    passengers = readr::parse_number(passengers, na = c("[422]", "[423]", "[424]", "NA")),
    year = as.character(year)
  )

# PKX Traffic wrangling
PKXTraffic <- PKXTraffic |>
  rename(
    year = Year,
    passengers = `Total passenger`
  ) |>
  select(
    year,
    passengers
  ) |>
  mutate(
    passengers = readr::parse_number(passengers, na = c(",", "[~]","NA")),
    year = substr(year, 1, 4),
    year = readr::parse_character(year, na = c("[~]", "NA"))
  )

# ICN Traffic wrangling
ICNTraffic <- ICNTraffic |>
  rename(
    year = Years,
    passengers = Passengers
  ) |>
  select(
    year,
    passengers
  ) |>
  slice_head(n=25) |>
  mutate(
    passengers = readr::parse_number(passengers, na = c(",", "NA")),
    year = as.character(year)
  )

# PVG traffic wrangling
PVGTraffic <- PVGTraffic |>
```

```r
  rename(
    year = Year,
    passengers = Passengers
  ) |>
  select(
    year,
    passengers
  ) |>
  mutate(
    year = str_remove_all(year, "\\[.*?\\]"),
    passengers = readr::parse_number(passengers, na = c(",", "NA"))
  )

# DXB traffic wrangling
DXBTraffic <- DXBTraffic |>
  rename(
    year = Year,
    passengers = Passengers
  ) |>
  select(
    year,
    passengers
  ) |>
  mutate(
    year = str_remove_all(year, "\\[.*?\\]"),
    passengers = readr::parse_number(passengers, na = c(",", "NA"))
  )

airport_traffic_20_to_24 <- ATLTraffic |>
  inner_join(FRATraffic, by = "year") |>
  inner_join(PKXTraffic, by = "year") |>
  inner_join(ICNTraffic, by = "year") |>
  inner_join(PVGTraffic, by = "year") |>
  inner_join(DXBTraffic, by = "year")

airport_traffic_20_to_24 <- airport_traffic_20_to_24 |>
  rename(
    ATL = passengers.x,
    FRA = passengers.y,
    PKX = passengers.x.x,
    ICN = passengers.y.y,
    PVG = passengers.x.x.x,
```

```r
    DXB = passengers.y.y.y
  ) |>
  slice_tail(n=5)

airport_graph_table <- airport_traffic_20_to_24 |>
  pivot_longer(
    cols = c("ATL", "FRA", "PKX", "ICN", "PVG", "DXB"),
    names_to = "airport",
    values_to = "passengers"
  ) |>
  mutate(
    passengers = round(passengers/1000000, digits = 2)
  )

airport_traffic_20_to_24 |>
  kable(
    col.names = c("Year", "Atlanta", "Frankfurt", "Beijing", "Incheon", "Shanghai", "Dubai")
    format.args = list(big.mark = ","),
    align = "c"
  ) |>
  kable_classic()
ggplot(
  data = airport_graph_table,
  mapping =  aes(x = year, y = passengers)
  ) +
  geom_bar(
    stat = "summary",
    fun = "sum",
    fill = "#112446"
  ) +
  labs(
    title = "Airport Traffic from 2020-2024",
    x = "Year",
    y = "# of Passengers (in hundered thousands)"
  ) +
  facet_wrap(vars(airport)) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5)
  )
library(tidyverse)
library(patchwork)
```

```r
# Monte carlo function that estimates the integration value
monte_carlo_integration <- function(f, n, xmin, xmax, ymin, ymax) {
  x <- runif(n, xmin, xmax)
  y <- runif(n, ymin, ymax)

  fx <- f(x)

  under_curve <- y <= fx

  rect_area <- (xmax - xmin) * (ymax - ymin)
  estimate <- mean(under_curve) * rect_area

  data.frame(
    x = x,
    y = y,
    fx = fx,
    under_curve = under_curve,
    n = n,
    estimate = estimate
  )
}

# population density beta distribution function
f <- function(x) {
  dbeta(x, shape1 = 2, shape2 = 2)
}

df10 <- monte_carlo_integration(f, 10, 0, 1, 0, 1.5)
df100 <- monte_carlo_integration(f, 100, 0, 1, 0, 1.5)
df1000 <- monte_carlo_integration(f, 1000, 0, 1, 0, 1.5)
df10000 <- monte_carlo_integration(f, 10000, 0, 1, 0, 1.5)


# Estimation for sample size n = 10
est10 <- df10 |>
  summarise(
    estimate = mean(under_curve) * 1.5
  ) |>
  pull(estimate)

# Plot for sample size n = 10
plot10 <- ggplot(
```

```r
  df10,
  aes(x, y, color = under_curve)
) +
  geom_point(
    alpha = 0.6,
    size = 1.5
  ) +
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
  ) +
  scale_color_manual(
    values = c("TRUE" = "green", "FALSE" = "red")
  ) +
  labs(
    title = "n = 10",
    subtitle = paste("Estimate =", round(est10, 4))
  ) +
  coord_cartesian(xlim = c(0,1), ylim = c(0, 1.5)) +
  theme_minimal() +
  theme(legend.position = "none")


# Estimation for sample size n = 100
est100 <- df100 |>
  summarise(
    estimate = mean(under_curve) * 1.5
  ) |>
  pull(estimate)

# Plot for sample size n = 100
plot100 <- ggplot(
  df100,
  aes(x, y, color = under_curve)
) +
  geom_point(
    alpha = 0.6,
    size = 1.5
  ) +
  stat_function(
    fun = dbeta,
```

```r
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
  ) +
  scale_color_manual(
    values = c("TRUE" = "green", "FALSE" = "red")
  ) +
  labs(
    title = "n = 100",
    subtitle = paste("Estimate =", round(est100, 4))
  ) +
  coord_cartesian(xlim = c(0,1), ylim = c(0, 1.5)) +
  theme_minimal() +
  theme(legend.position = "none")


# Estimation for sample size n = 1,000
est1000 <- df1000 |>
  summarise(
    estimate = mean(under_curve) * 1.5
  ) |>
  pull(estimate)

# Plot for sample size n = 1,000
plot1000 <- ggplot(
  df1000,
  aes(x, y, color = under_curve)
) +
  geom_point(
    alpha = 0.6,
    size = 1.5
  ) +
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
  ) +
  scale_color_manual(
    values = c("TRUE" = "green", "FALSE" = "red")
  ) +
  labs(
    title = "n = 1000",
    subtitle = paste("Estimate =", round(est1000, 4))
```

```r
  ) +
  coord_cartesian(xlim = c(0,1), ylim = c(0, 1.5)) +
  theme_minimal() +
  theme(legend.position = "none")


# Estimation for sample size n = 10,000
est10000 <- df10000 |>
  summarise(
    estimate = mean(under_curve) * 1.5
  ) |>
  pull(estimate)

# Plot for sample size n = 10,000
plot10000 <- ggplot(
  df10000,
  aes(x, y, color = under_curve)
) +
  geom_point(
    alpha = 0.6,
    size = 1.5
  ) +
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
  ) +
  scale_color_manual(
    values = c("TRUE" = "green", "FALSE" = "red")
  ) +
  labs(
    title = "n = 10000",
    subtitle = paste("Estimate =", round(est10000, 4))
  ) +
  coord_cartesian(xlim = c(0,1), ylim = c(0, 1.5)) +
  theme_minimal() +
  theme(legend.position = "none")

# Generates small multiple visualization of the four graphs for each sample size
plot10 + plot100 + plot1000 + plot10000
# Load tidyverse
library(tidyverse)
```

```r
# 1. Load data
calcium <- read_csv("calcium.csv")

# 2. Rename columns
colnames(calcium) <- c(
  "initialA", "year1A", "year2A", "year3A",
  "initialB", "year1B", "year2B", "year3B"
)

# 3. Add participant ID
calcium <- calcium %>%
  mutate(id = row_number())

# 4. Pivot to long format
calcium_long <- calcium %>%
  pivot_longer(
    cols = -id,
    names_to = c("time", "treatment_group"),
    names_pattern = "(initial|year1|year2|year3)(A|B)",
    values_to = "calcium"
  ) %>%
  drop_na(calcium)

# 5. Clean and format variables
calcium_long <- calcium_long %>%
  mutate(
    time = factor(time, levels = c("initial", "year1", "year2", "year3")),
    treatment_group = factor(treatment_group)
  )

# 6. Plot individual trajectories
ggplot(calcium_long, aes(x = time, y = calcium, group = interaction(id, treatment_group))) +
  geom_line(alpha = 0.4) +
  geom_point() +
  facet_wrap(~ treatment_group) +
  labs(
    title = "Ulnar Calcium Measurements Over Time by Treatment Group",
    x = "Time",
    y = "Calcium Measurement"
  ) +
  theme_minimal()
```